SMC Similarity (Decimal → Bit-Level)

```
def decimal_to_binary_list(nums, bit_length):
    bin_list = []
    for num in nums:
        b = bin(num)[2:].zfill(bit_length)  # Convert to binary and pad with
zeros
        bin_list.extend([int(bit) for bit in b])
    return bin_list

# -------- User Input --------
x = list(map(int, input("Enter decimal numbers for vector X: ").split()))
y = list(map(int, input("Enter decimal numbers for vector Y: ").split()))

# Determine bit length
max_num = max(max(x), max(y))
bit_length = max_num.bit_length()

# Convert to bit-level binary vectors
x_bin = decimal_to_binary_list(x, bit_length)
y_bin = decimal_to_binary_list(y, bit_length)

# -------- Compute SMC --------
q = r = s = t = 0
for a, b in zip(x_bin, y_bin):
    if a == 1 and b == 1:
        q += 1
    elif a == 1 and b == 0:
        r += 1
    elif a == 0 and b == 1:
        s += 1
    elif a == 0 and b == 0:
        t += 1

smc_similarity = (q + t) / (q + r + s + t)

print(f"\nBit-level vector X: {x_bin}")
print(f"Bit-level vector Y: {y_bin}")
print(f"SMC Similarity (bit-level): {smc_similarity:.4f}")
```

Jaccard Similarity (Decimal → Bit-Level

```
def decimal_to_binary_list(nums, bit_length):
    bin_list = []
    for num in nums:
        b = bin(num)[2:].zfill(bit_length)  # Convert to binary and pad with
zeros
        bin_list.extend([int(bit) for bit in b])
    return bin_list

# -------- User Input --------
x = list(map(int, input("Enter decimal numbers for vector X: ").split()))
y = list(map(int, input("Enter decimal numbers for vector Y: ").split()))

# Determine bit length
max_num = max(max(x), max(y))
bit_length = max_num.bit_length()

# Convert to bit-level binary vectors
x_bin = decimal_to_binary_list(x, bit_length)
y_bin = decimal_to_binary_list(y, bit_length)

# -------- Compute Jaccard Similarity --------
q = r = s = 0
for a, b in zip(x_bin, y_bin):
    if a == 1 and b == 1:
        q += 1
    elif a == 1 and b == 0:
        r += 1
    elif a == 0 and b == 1:
        s += 1

denom = q + r + s
jaccard_similarity = q / denom if denom != 0 else 0.0

print(f"\nBit-level vector X: {x_bin}")
print(f"Bit-level vector Y: {y_bin}")
print(f"Jaccard Similarity (bit-level): {jaccard_similarity:.4f}")
```

Cosine Similarity (Decimal Input)

```
import math

# -------- User Input --------
x = list(map(float, input("Enter elements of vector X: ").split()))
y = list(map(float, input("Enter elements of vector Y: ").split()))

# Check if vectors are the same length
if len(x) != len(y):
    print("Error: Vectors must be the same length.")
    exit()

# -------- Compute Cosine Similarity --------
dot_product = sum(a * b for a, b in zip(x, y))
norm_x = math.sqrt(sum(a * a for a in x))
norm_y = math.sqrt(sum(b * b for b in y))

# Handle zero-vector cases
if norm_x == 0 or norm_y == 0:
    cosine_similarity = 0.0
else:
    cosine_similarity = dot_product / (norm_x * norm_y)

print(f"\nVector X: {x}")
print(f"Vector Y: {y}")
print(f"Cosine Similarity: {cosine_similarity:.4f}")
```

Tanimoto Coefficient (Decimal Input)

```python
# -------- User Input --------
x = list(map(float, input("Enter elements of vector X: ").split()))
y = list(map(float, input("Enter elements of vector Y: ").split()))

# Check if vectors are the same length
if len(x) != len(y):
    print("Error: Vectors must be the same length.")
    exit()

# -------- Compute Tanimoto Coefficient --------
dot_xy = sum(a * b for a, b in zip(x, y))
dot_xx = sum(a * a for a in x)
dot_yy = sum(b * b for b in y)

denom = dot_xx + dot_yy - dot_xy
if denom == 0:
    tanimoto = 0.0
else:
    tanimoto = dot_xy / denom

print(f"\nVector X: {x}")
print(f"Vector Y: {y}")
print(f"Tanimoto Coefficient: {tanimoto:.4f}")
```

Assignment 2: finding gain and best attribute:

Age	Income	Status	Credit Rating	Buy Smart Phone
Mid	Fair	Employee	Average	Yes
Mid	Fair	Employee	Max	No
Young	Low	Student	Min	Yes
Old	High	Employee	Max	No
Old	Fair	Retire	Average	No
Young	Low	Student	Min	Yes
Young	Fair	Employee	Average	Yes
Mid	Low	Employee	Max	No
Young	Fair	Student	Average	Yes
Old	High	Retire	Min	No
Young	Fair	Employee	Average	Yes
Mid	Low	Employee	Max	Yes

```
import math

# -------- DATASET --------
data = [
    ['Mid', 'Fair', 'Employee', 'Average', 'Yes'],
    ['Mid', 'Fair', 'Employee', 'Max', 'No'],
    ['Young', 'Low', 'Student', 'Min', 'Yes'],
    ['Old', 'High', 'Employee', 'Max', 'No'],
    ['Old', 'Fair', 'Retire', 'Average', 'No'],
    ['Young', 'Low', 'Student', 'Min', 'Yes'],
    ['Young', 'Fair', 'Employee', 'Average', 'Yes'],
    ['Mid', 'Low', 'Employee', 'Max', 'No'],
    ['Young', 'Fair', 'Student', 'Average', 'Yes'],
    ['Old', 'High', 'Retire', 'Min', 'No'],
    ['Young', 'Fair', 'Employee', 'Average', 'Yes'],
    ['Mid', 'Low', 'Employee', 'Max', 'Yes']
]

features = ['Age', 'Income', 'Status', 'Credit']
```

```python
# -------- ENTROPY FUNCTION --------
def entropy(dataset):
    total = len(dataset)
    yes = sum(1 for row in dataset if row[-1] == 'Yes')
    no = total - yes

    def calc(p):
        return -p * math.log2(p) if p != 0 else 0

    return calc(yes/total) + calc(no/total)

# -------- INFORMATION GAIN --------
def information_gain(dataset, index):
    total_entropy = entropy(dataset)
    values = set(row[index] for row in dataset)

    weighted_entropy = 0
    print(f"\nFeature: {features[index]}")

    for v in values:
        subset = [row for row in dataset if row[index] == v]
        weight = len(subset) / len(dataset)
        sub_entropy = entropy(subset)

        weighted_entropy += weight * sub_entropy

        print(f"  Value: {v}")
        print(f"    Count = {len(subset)}")
        print(f"    Entropy = {round(sub_entropy, 3)}")

    gain = total_entropy - weighted_entropy
    print(f"  → Information Gain = {round(gain, 3)}")

    return gain

# -------- MAIN --------
print("Entropy of dataset:", round(entropy(data), 3))

gains = []
print("\n--- Information Gain Calculation ---")

for i in range(len(features)):
    g = information_gain(data, i)
    gains.append(g)
```

```python
# -------- BEST ATTRIBUTE --------
best_index = gains.index(max(gains))
print("\nBest Attribute:", features[best_index])
```